EPITA International Programs | AI Project Methodology 2026

## Graded Project, Part 2

# AI Project Technical Methodologies

Building the RetailGenius churn model with production-ready practices

**Group:** Group 5

**Members:** Aravind Reddy Kolan, Prem Siva Naga Sai Ram Kumar Gudivada, Praveen Kumar Koppula

**Date:** November 2026

**GitHub repository:** https://github.com/premgudivada7/ai-project-methodolagy- (shared with dinamedy@hotmail.com)

# Contents


## 1. Introduction

## 2. Data

- 2.1 The dataset
- 2.2 What we found during exploration
- 2.3 A data leakage trap: duplicated customers

## 3. Project structure

## 4. Engineering best practices

- 4.1 Versioning and collaboration
- 4.2 Standardisation
- 4.3 Reproducibility
- 4.4 Documentation and tests

## 5. MLflow

- 5.1 Experiment tracking
- 5.2 MLflow Projects
- 5.3 Model Registry
- 5.4 Serving
- 5.5 Bonus: cloud deployment

## 6. Results

## 7. Limits and next steps

## 8. How to reproduce

## References

# 1. Introduction

In Part 1 we framed the churn project for RetailGenius from a business point of view. In this part we build it. We wanted the project to look like something a real team could pick up and run in production, so we put as much effort into the structure and tooling as into the model. That means Git and GitHub, a cookiecutter layout, PEP8 checks, locked dependencies, generated documentation and MLflow for the whole model lifecycle.

As the assignment says, getting the best possible score was not the goal. We still compared several models, and the results turned out to be good. We found one data problem along the way (duplicated customers) that is worth explaining, because it would have made our numbers look better than they really are.

All the code is in our GitHub repository: **https://github.com/premgudivada7/ai-project-methodolagy-**. The repository is shared with dinamedy@hotmail.com.

# 2. Data

## 2.1 The dataset

We used the E-Commerce Customer Churn dataset from Kaggle. The Excel file has a data dictionary sheet and an "E Comm" sheet with 5,630 customers and 20 columns. The target is **Churn** (1 = the customer left). The other columns describe the customer (tenure, city tier, marital status, gender), how they shop (preferred device, payment mode, order category, order count, coupons, cashback, days since last order) and how they feel about the service (satisfaction score, complaints).

## 2.2 What we found during exploration

The exploration lives in **notebooks/01_eda.ipynb**. The main findings shaped the cleaning code:

| Finding | What we did about it |
|---|---|
| 16.8% of customers churn, so the classes are imbalanced. | Stratified split, balanced class weights, and recall / PR-AUC as main metrics instead of accuracy. |
| Seven numeric columns have about 5% missing values (Tenure, WarehouseToHome, HourSpendOnApp, OrderAmountHikeFromlastYear, CouponUsed, OrderCount, DaySinceLastOrder). | Median imputation inside the model pipeline, fitted on the training data only, so the test set does not leak into it. |
| The same thing has different labels: "Phone" and "Mobile Phone", "CC" and "Credit Card", "COD" and "Cash on Delivery", "Mobile" and "Mobile Phone". | Mapped to one label each in churn.dataset.clean(). |
| 557 rows are exact copies of another row except for the CustomerID (after fixing the labels). | Removed. See section 2.3. |
| Gender is in the data but has no business reason to drive a retention decision. | Excluded from the model inputs (data minimisation, fairness). It stays in the data for fairness checks. |

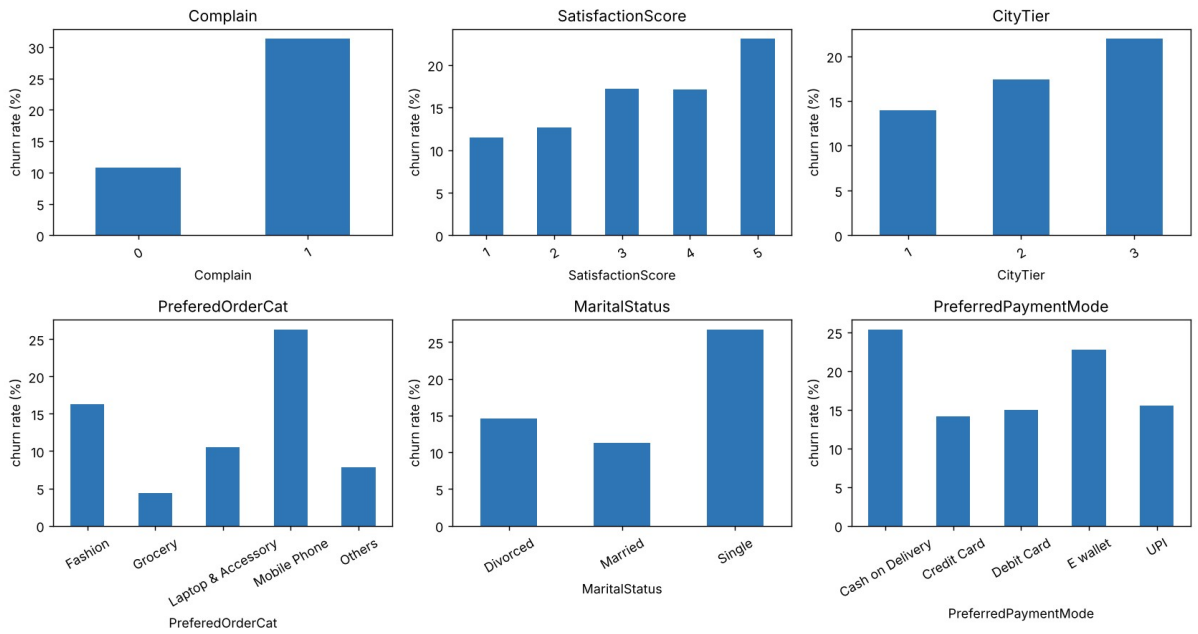



*Figure 1 – Churn rate by category (notebooks/01_eda.ipynb)*

Some patterns are clear even before modelling. Customers who complained churn about three times more often (31% vs 11%). Single customers churn more than married ones, and so do customers who mostly buy mobile phones or pay cash on delivery. One surprise is that churn goes **up** with the satisfaction score. That looks wrong at first, but it probably means the survey is answered by people who are about to leave, or that the score is coded differently from what we assumed. We flagged it as a question for the business rather than "fixing" it.

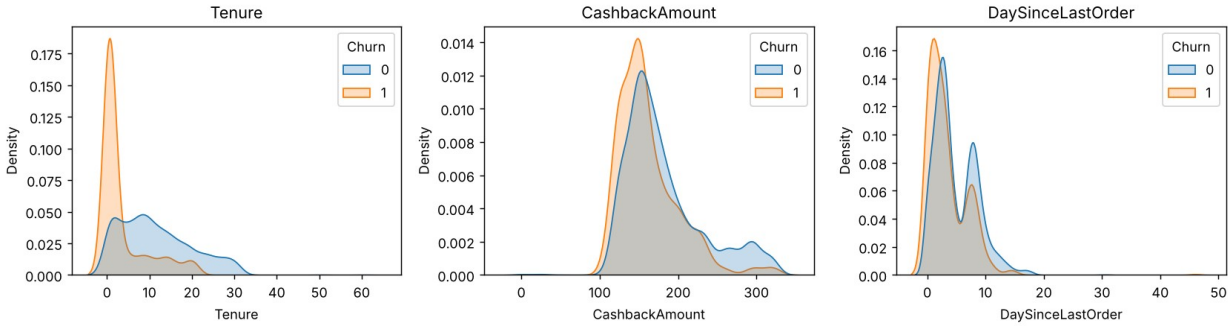



*Figure 2 – Distributions of tenure, cashback and days since last order, churners (1) vs others (0)*

Tenure is the strongest single signal. Two thirds of the churners have a tenure of one month or less, while customers who have been around for more than a year churn only 5% of the time. Churners also tend to have lower cashback.

## 2.3 A data leakage trap: duplicated customers

Our first models scored a ROC-AUC of 0.998, which is suspicious for a churn problem. When we looked closer, 557 rows were identical to another row on every column except the ID. After a random split, **193 of the 1,126 test customers had an exact twin in the training set**. A deep random forest can memorise those twins, so the test score was partly measuring memory, not prediction.

We now drop these copies in the cleaning step, before the split. This left 5,073 customers (churn rate 16.6%) and brought the best ROC-AUC down from 0.998 to about 0.985. The scores are still high, partly because the dataset seems to contain more near-duplicates that we cannot detect with certainty. We mention this as a limit in section 7.

## 3. Project structure

We started from the **cookiecutter-data-science** template (DrivenData) and adapted it to our needs. The idea is that anyone opening the repository knows where to find things: raw data is never modified, generated data can be rebuilt at any time, and the source code is a real Python package rather than a folder of notebooks.

```
ai-project-methodology-/
├── MLproject, conda.yaml      MLflow Project + its environment
├── pyproject.toml             poetry dependencies, black and pylint settings
├── requirements*.txt          the same dependencies for pip
├── Makefile                   one command per step (make data, make experiments...)
├── Dockerfile                 serving image for cloud deployment
├── .github/workflows/ci.yml   black + flake8 + pytest on every push
├── data/raw | interim | processed
├── churn/                     the Python package
│   ├── config.py              paths, column lists, MLflow settings
│   ├── dataset.py             1. data preparation
│   ├── features.py            2. feature engineering
│   ├── train.py               3. training + MLflow tracking
│   ├── experiments.py            several runs to compare
│   ├── registry.py            4. Model Registry
│   ├── predict.py             5. inference
│   └── explain.py             6. SHAP (Part 3)
├── notebooks/01_eda.ipynb
├── scripts/query_server.py    client for the REST server
├── tests/                     pytest unit tests
├── docs/                      Sphinx documentation
├── models/                    trained models (.joblib)
├── mlflow.db, mlruns/         MLflow store and artifacts
└── reports/                   the three reports, figures, metrics
```

The workflow is split into separate scripts, one per step, as asked. Each script reads the output of the previous one from disk, so a step can be re-run on its own.

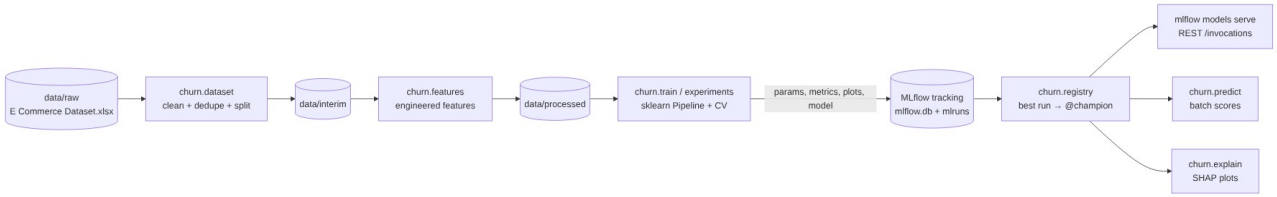



*Figure 3 – Pipeline steps and where MLflow comes in*

| Step | Module | What it does |
|---|---|---|
| Data preparation | churn/dataset.py | Reads the Excel sheet, checks the schema, harmonises labels, drops duplicates, writes a data quality report and a stratified 80/20 train/test split. |
| Feature engineering | churn/features.py | Adds CashbackPerOrder, CouponPerOrder and IsNewCustomer (tenure of one month or less). Defines the preprocessing (median / most-frequent imputation, one-hot encoding, scaling for the linear model) as a ColumnTransformer. |
| Training | churn/train.py | Builds a scikit-learn Pipeline (preprocessing + classifier), runs 5-fold cross-validation, fits, evaluates on the test set and logs everything to MLflow. Supports logistic regression, random forest, gradient boosting and XGBoost. |
| Inference | churn/predict.py | Loads the champion model from the registry (or a local file), applies the same cleaning and features to new customers and outputs a probability, a 0/1 flag and a Low / Medium / High risk segment. |

Keeping preprocessing inside the Pipeline was an important choice. The model saved in MLflow contains the imputers and encoders, so whoever calls it (batch job, REST server, SHAP) sends raw columns and gets the exact same transformations as during training.

# 4. Engineering best practices

## 4.1 Versioning and collaboration

- All code, configuration, the raw dataset, MLflow outputs and reports are versioned in Git and hosted on GitHub.
- We work on short-lived feature branches (feature/data-prep, feature/mlflow, feature/shap…) merged into main through pull requests, with at least one review from another group member.
- A GitHub Actions workflow (.github/workflows/ci.yml) runs black, flake8 and the unit tests on every push and pull request, so broken or badly formatted code is caught before it reaches main.
- Models are versioned twice: as MLflow runs (each with its Git commit, parameters and the MD5 hash of the training file) and as numbered versions in the MLflow Model Registry.

## 4.2 Standardisation

The whole group uses **Python 3.11** (pinned in .python-version and conda.yaml). Code follows PEP8 and is checked with three tools, all configured for a line length of 100:

| Tool | Role | Command |
|---|---|---|
| black | Formats the code automatically, so we never argue about style | make format |
| flake8 | PEP8 errors, unused imports, undefined names | make lint |
| pylint | Deeper checks (naming, complexity, docstrings), must score at least 8/10 | make lint |

Every module and public function has a Google-style docstring. Shared settings (paths, column names, random seed, MLflow names) live in churn/config.py instead of being repeated across scripts.

## 4.3 Reproducibility

- **poetry** is our main dependency manager (pyproject.toml with a lock file). For people who prefer conda, conda.yaml builds the same environment, and requirements.txt / requirements-dev.txt cover plain pip.
- A fixed random seed (42) is used for the split, cross-validation and the models.
- The MLproject file declares the environment, so mlflow run rebuilds it before running any step.
- Each MLflow run stores its Git commit, parameters, data hash and the exact library versions of the logged model.
- A Makefile wraps every step, so the full pipeline is one command: make all.

## 4.4 Documentation and tests

The code documentation is generated with **Sphinx** (autodoc + napoleon) from the docstrings. make docs builds the HTML site in docs/build/html. MLflow and SHAP are mocked during the build, so the docs can be generated without installing them.

*[ Screenshot: Sphinx HTML documentation, page "churn package" (docs/build/html/index.html) ]*

The unit tests in tests/ use a small synthetic dataset with the same columns as the real one. They check schema validation, label harmonisation, duplicate removal, the stratified split, feature engineering (no division by zero), that Gender is never a model input, an end-to-end fit and score, and the metric functions.

# 5. MLflow

## 5.1 Experiment tracking

Every call to churn.train opens an MLflow run in the experiment **retailgenius-churn**. The tracking store is a local SQLite database (mlflow.db) and artifacts go to mlruns/. Both are committed to the repository so the results can be checked without retraining. For each run we log:

| Type | Content |
|---|---|
| Parameters | model type, n_estimators, max_depth, min_samples_leaf, learning_rate, C, class_weight, decision threshold |
| Metrics | cross-validated ROC-AUC and PR-AUC (mean and std over 5 folds); on the test set: accuracy, precision, recall, F1, ROC-AUC, PR-AUC, Brier score, lift in the top 10%, best F1 threshold |
| Tags | model type, MD5 of the training file, train / test sizes, Git commit (added automatically by MLflow) |
| Artifacts | confusion matrix, ROC and precision-recall curves, feature importances, the model itself with its signature and an input example |
| Dataset | the training dataframe, registered with mlflow.log_input |

churn.experiments trains seven configurations in one go (two logistic regressions, three random forests and two gradient boosting models), which gives us several runs to compare side by side in the UI.

```
make experiments          # 7 runs
make ui                   # mlflow ui --backend-store-uri sqlite:///mlflow.db  ->
http://127.0.0.1:5000
```

*[ Screenshot: MLflow UI, experiment "retailgenius-churn", table of the 7 runs with metrics columns ]*

*[ Screenshot: MLflow UI, "Compare runs" view (parallel coordinates or scatter of cv_pr_auc_mean vs n_estimators) ]*

*[ Screenshot: MLflow UI, one run with its parameters, metrics and plots/ artifacts ]*

## 5.2 MLflow Projects

The MLproject file at the root turns the repository into an MLflow Project. It points to conda.yaml for the environment and declares one entry point per step, plus main, which chains them all.

| Entry point | Runs | Parameters |
|---|---|---|
| prepare | python -m churn.dataset | – |
| features | python -m churn.features | – |
| train | python -m churn.train | model, n_estimators, max_depth, min_samples_leaf, learning_rate, C, class_weight, threshold |
| experiments | python -m churn.experiments | – |
| register | python -m churn.registry | metric (default cv_pr_auc_mean) |
| explain | python -m churn.explain | – |
| main | all of the above in order | – |

```
mlflow run . -e main --env-manager local
mlflow run . -e train -P model=gb -P n_estimators=300 -P max_depth=4 --env-manager local
# from GitHub directly:
mlflow run https://github.com/premgudivada7/ai-project-methodolagy- -e main
```

### 5.3 Model Registry

churn.registry picks the best finished run and registers its model as a new version of **retailgenius-churn-model**. We choose on the **cross-validated** PR-AUC, not the test score. Picking the run with the best test score would quietly turn the test set into a validation set and make our final numbers optimistic.

The selected version gets the alias **@champion**, a description with its scores and a tag validation_status = approved. Serving and inference always load models:/retailgenius-churn-model@champion, so promoting a new model or rolling back is just a matter of moving the alias. We use aliases instead of the older Staging / Production stages, which MLflow has deprecated.

```
make register              # register best run, set @champion
python -m churn.registry --list
```

*[ Screenshot: MLflow UI, "Models" tab, retailgenius-churn-model with its versions and the @champion alias ]*

### 5.4 Serving

The champion model is served locally with MLflow's built-in REST server. Because we logged the model with pyfunc_predict_fn="predict_proba", the server returns churn probabilities rather than only 0/1 labels. scripts/query_server.py sends three test customers and prints the answer.

```
make serve   # mlflow models serve -m "models:/retailgenius-churn-model@champion" -p 5001 --
env-manager local
make query   # POST http://127.0.0.1:5001/invocations  {"dataframe_split": {...}}
```

*[ Screenshot: terminal with the server running, and the output of make query (probabilities for 3 customers) ]*

### 5.5 Bonus: cloud deployment

The Dockerfile packages the tracking store and the champion model into an image that runs mlflow models serve on port 5001. The same image can go to any container service. Our target was Azure Container Apps (free student credits):

```
docker build -t retailgenius-churn .
docker run -p 5001:5001 retailgenius-churn            # local test
az acr build -r <registry> -t retailgenius-churn .     # push to Azure Container Registry
az containerapp up -n churn-api -g <rg> --image <registry>.azurecr.io/retailgenius-churn --
target-port 5001 --ingress external
```

*[ If deployed: public URL of the endpoint and a screenshot of a request to it ]*

## 6. Results

All numbers below are on the same held-out test set of 1,015 customers (after removing duplicates). "CV PR-AUC" is the mean over 5 folds on the training set and is what we used to choose the model.

| Run | CV PR-AUC | ROC-AUC | PR-AUC | Recall | Precision | F1 | Lift top 10% |
|---|---|---|---|---|---|---|---|
| rf_500_full | 0.910 | 0.985 | 0.952 | 0.774 | 0.963 | 0.858 | 6.0× |
| gb_400_d5 | 0.908 | 0.980 | 0.955 | 0.857 | 0.929 | 0.892 | 6.0× |
| rf_300_d12 | 0.886 | 0.978 | 0.927 | 0.756 | 0.914 | 0.827 | 5.8× |

| Run | CV PR-AUC | ROC-AUC | PR-AUC | Recall | Precision | F1 | Lift top 10% |
|---|---|---|---|---|---|---|---|
| gb_200_d3 | 0.844 | 0.950 | 0.859 | 0.649 | 0.886 | 0.749 | 5.5× |
| rf_100_d6 | 0.753 | 0.911 | 0.756 | 0.768 | 0.535 | 0.631 | 5.1× |
| logreg_C0.1 | 0.730 | 0.888 | 0.690 | 0.839 | 0.465 | 0.599 | 4.5× |
| logreg_C1 | 0.729 | 0.889 | 0.685 | 0.845 | 0.458 | 0.594 | 4.4× |

The tree ensembles clearly beat the logistic regression baseline, which tells us the relationships are not linear (tenure and cashback, for example, have threshold effects). The random forest with 500 fully grown trees has the best cross-validated PR-AUC (0.910, just ahead of gradient boosting at 0.908), so our selection rule makes it the champion. Its ranking is excellent: if marketing only contacted the 10% riskiest customers, that group would churn about 6 times more often than the average customer.

At the default 0.5 threshold, though, the random forest is cautious. **96% of the customers it flags really churn, but it only catches 77% of the churners.** Since Part 1 made recall the priority (missing a churner costs more than an unnecessary offer), 0.5 is not the right threshold for this model. Its best F1 threshold on the test set is 0.35, which raises F1 from 0.858 to 0.896. In production we would set this threshold with the business, using the --threshold option of churn.train, and choose it on validation data rather than the test set.

The gradient boosting model is a very close second. It has a higher recall at 0.5 (0.857) and the best calibrated probabilities (Brier score 0.028 vs 0.041). The gap in cross-validation is so small that either model would be a reasonable choice. We kept the rule-based pick, because changing it after looking at the test results would be exactly the kind of test-set tuning we wanted to avoid.

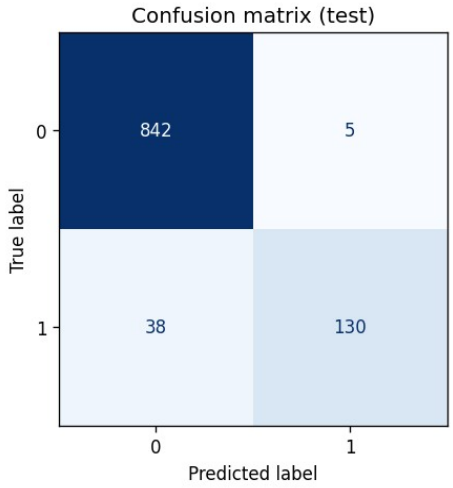



*Figure 4 – Confusion matrix of the champion model on the test set (threshold 0.5)*

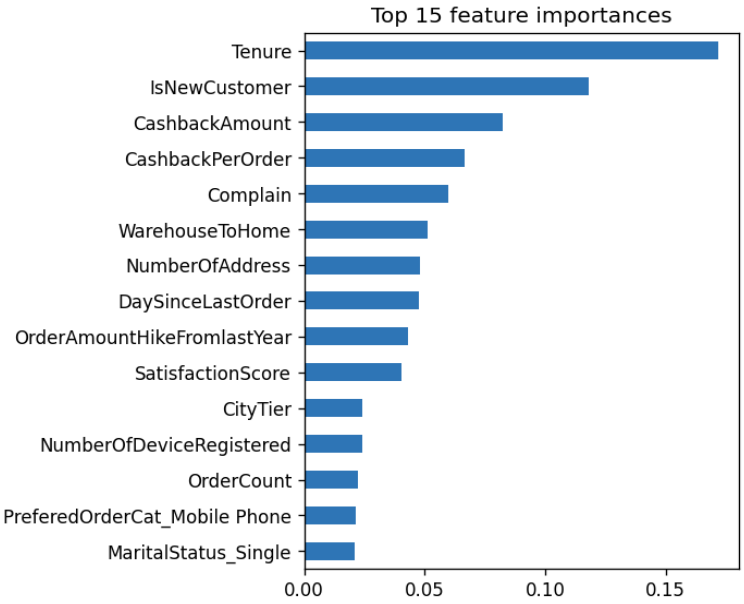



*Figure 5 – Top 15 feature importances (impurity based) of the champion model*

Tenure and our engineered IsNewCustomer feature dominate, followed by cashback, complaints and distance to the warehouse. This matches what we saw in the exploration. Impurity-based importances only say how much a feature is used, not in which direction it pushes the prediction. That is what SHAP adds in Part 3.

## 7. Limits and next steps

- **Scores are probably optimistic.** Besides the exact duplicates we removed, the dataset likely contains near-duplicates (the same customer with slightly different values). Real-life churn models usually land well below a ROC-AUC of 0.98. On real RetailGenius data we would also use a time-based split instead of a random one.
- **The satisfaction score goes the "wrong" way.** Higher scores come with more churn. We would check with the business how the survey is collected before using this feature in production.
- **No real churn window.** The Kaggle target is already given. At RetailGenius we would build the label ourselves, as described in Part 1 (no order for 90 days).
- **Next steps:** hyper-parameter search with Optuna, probability calibration, monitoring of data drift (e.g. Evidently), and a scheduled batch scoring job.

## 8. How to reproduce

```
git clone https://github.com/premgudivada7/ai-project-methodolagy-.git
cd ai-project-methodolagy-
poetry install && poetry shell          # or: conda env create -f conda.yaml
make experiments                        # data + features + 7 MLflow runs
make register                           # @champion in the Model Registry
make ui                                 # http://127.0.0.1:5000
make serve   # then, in another terminal: make query
make explain                            # Part 3
make docs && make test && make lint
```

## References


[1] DrivenData, Cookiecutter Data Science. https://cookiecutter-data-science.drivendata.org/

[2] MLflow documentation: Tracking, Projects, Model Registry, Model Serving. https://mlflow.org/docs/latest/ml/

[3] scikit-learn documentation: Pipeline, ColumnTransformer, model evaluation. https://scikit-learn.org/stable/

[4] PEP 8 – Style Guide for Python Code. https://peps.python.org/pep-0008/

[5] Poetry documentation. https://python-poetry.org/docs/

[6] Sphinx documentation (autodoc, napoleon). https://www.sphinx-doc.org/

[7] Kaggle, E-Commerce Customer Churn Analysis and Prediction. https://www.kaggle.com/datasets/ankitverma2010/ecommerce-customer-churn-analysis-and-prediction

[8] Kaufman, S. et al. (2012). Leakage in data mining: formulation, detection, and avoidance. ACM TKDD.

[9] Microsoft Learn, Azure Container Apps quickstart. https://learn.microsoft.com/azure/container-apps/

[10] Generative AI (Claude, Anthropic) was used to help write code and parts of this report. The group reviewed, ran and edited everything.